

Joint-Embedding Prediction over Scientific Reasoning

Stages for Efficient Language Model

Audience	Farhana Keya
Maintainer	Gollam Rabby, Soren Auer
Version	0.0
Last updated	
Status	Phase 1 (fine-tuning) — active

How to read this document

This document is **complete on purpose**. Read **Parts 0–3 carefully once** (the ideas and how to run). Then keep it open and use **Parts 4–6 as a lookup** when coding, modifying, or debugging. Every claim about cost or quality is written honestly — when something is approximate, risky, or not-yet-implemented, it says so.

Table of contents

- **Part 0 — The research plan** (start here; explains *why* before *how*)
 - **Part 1 — The idea** (JEPA, our scientific-chain twist, the metrics)
 - **Part 2 — The code** (file map, switches, data flow)
 - **Part 3 — Running experiments** (setup, smoke test, the three studies)
 - **Part 4 — Modifying the code** (recipes for common changes)
 - **Part 5 — Troubleshooting** (errors and fixes)
 - **Part 6 — Reference** (config table, glossary, checklists)
-

Part 0 — The Research Plan

0.1 What question are we answering?

We test **one claim**:

Adding a JEPA objective to a language model produces a stronger generative model than the standard next-token objective alone.

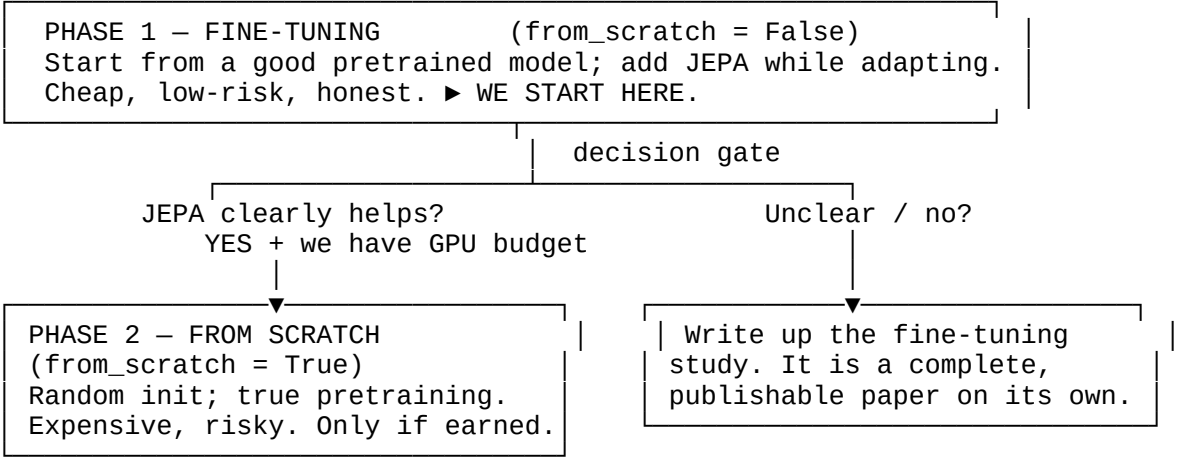
We measure "stronger generative" with **perplexity** (lower = the model predicts/generates text better). The whole project reduces to one comparison:

$\text{Perplexity}(\text{NTP} + \text{JEPA}) < ? \text{Perplexity}(\text{NTP alone})$

Everything else — ablations, sweeps — explains *why* and *how much*.

0.2 The two-phase plan (the single most important thing to understand)

There are two regimes in which we can test the claim. They differ enormously in cost and risk, so we do them **in order**, not together.



	Phase 1: Fine-tuning	Phase 2: From scratch
Switch	from_scratch = False	from_scratch = True
Starting weights	A downloaded, already-good model	Random noise
Tests	"Does JEPA help <i>adapting</i> a model?"	"Does JEPA help <i>pretraining</i> a model?"
Compute	Hours, 1–2 GPUs	Days, many GPUs
Risk	Low	High (may not converge)
Data needs	Our corpus suffices	Needs far more than we have
Claim honesty	"JEPA helps fine-tuning" — easy	"JEPA helps pretraining" — hard at our scale

The strategy in one line: *Prove the idea cheaply in Phase 1; spend the big compute on Phase 2 only if Phase 1 earns it.* You start with `from_scratch = False`. Do **not** flip it until we pass the gate together.

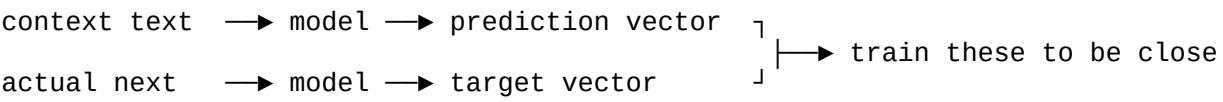
Part 1 — The Idea

1.1 JEPA in plain language

JEPA = *Joint-Embedding Predictive Architecture*. The core idea:

Instead of predicting the *exact next words*, predict the *embedding* (meaning vector) of what comes next.

A standard language model does **next-token prediction (NTP)**: given text, predict the probability of each next word — it must get every surface detail right. JEPA instead operates in *meaning space*:



1. Take a **context** (some text).
2. The model produces a **prediction vector** for what comes next.
3. The actual next thing is **encoded** into a **target vector**.
4. Train so prediction $\approx$ target (high cosine similarity).

1.2 Why we combine JEPA with NTP

JEPA **alone** can cheat: make every vector identical, and prediction always "matches." The embeddings become useless — this failure is called **representation collapse**. NTP prevents it, because predicting real words forces embeddings to stay informative. So our objective is **both**:

$L = \text{predict real words (prevents collapse)} + \lambda \cdot \text{predict meaning vectors (JEPA)}$

The weight λ controls how much JEPA matters. We study λ directly (Part 3).

1.3 Our twist: papers as reasoning chains

Generic JEPA predicts "the next chunk of text." We make it **structured and scientific**. Each paper is a **6-stage reasoning chain**:

title $\rightarrow$ hypothesis $\rightarrow$ method $\rightarrow$ experiment $\rightarrow$ result $\rightarrow$ conclusion

The JEPA objective becomes: **predict the embedding of a later stage from the earlier stages**.

Which stage predicts which is defined by **edges**:

chain_edges = [(0,1), (1,2), (2,3), (3,4), (4,5), (2,4)]
 # — adjacent "next-stage" edges — $\perp$ long-range: method $\rightarrow$ result

The five adjacent edges are "predict the next step." The extra (2,4) edge tests **deeper reasoning**: can the model anticipate the *result* from the *method* alone?

The full objective, precisely (only edges *not* skipped by loss dropout, set E' , contribute):

$L = L_{\text{NTP}} + \lambda \cdot |E'| \sum_{(i,j) \in E'} (1 - \cos(\text{pred}_i, \text{target}_j))$

1.4 The metrics (what "good" means)

Metric	Question it answers	How
Perplexity (headline)	Is it a good generator?	$\exp(\text{avg next-token loss})$; lower is better
F1 — Forward Retrieval	Does it understand findings, not just words?	Given 4 stages, pick the <i>real</i> result from a lineup containing an adversarial fake (a flipped finding, e.g. "increased" $\rightarrow$ "decreased"). Reported as Recall@1/5, MRR.
F2 — Surprise Calibration	Is the model's surprise meaningful?	Does <i>surprise</i> at the real result (1 – cosine) correlate with citations ? Reported as Spearman ρ , AUROC.

Mental model: Perplexity is the headline (generation quality). F1 checks *understanding* (a flipped finding fools a word-matcher but not a true understander). F2 is an interpretive bonus (surprise vs. impact).

Part 2 — The Code

2.1 File map

config.py	← ALL settings live here. Edit this, not the others.
▲	
utils.py	← tiny helpers (parse year, get field)
balance.py	← citation labels for F2 (handles citation skew)
distractors.py	← builds the F1 candidate lineup (incl. the adversarial fake)
data_stream.py	← streams the dataset → Parquet shards → DataLoader
model.py	← the model: pretrained OR random init; NTP + JEPA methods
train.py	← the training loop (trains ONE model)
evaluate.py	← perplexity + F1 + F2 (evaluates ONE model)
stats.py	← significance tests (paired t-test, Holm-Bonferroni)
experiments.py	← ★ THE ENTRY POINT ★ runs the studies (calls train + evaluate)

File	When you'll touch it
config.py	Almost everything: model, data size, hyperparameters, switches.
experiments.py	The studies and how they sweep settings. This is what you run.
model.py	If you change the architecture or the JEPA loss.
data_stream.py	If you change the dataset or the chain definition.
The rest	Rarely.

You only ever run experiments.py. It calls `train()` and `evaluate()` internally. You do *not* run `train.py` or `evaluate.py` by hand (except when debugging a single model).

2.2 The control switches (the heart of everything)

These live in `config.py`. Understanding these five *is* understanding the project. An experiment = **flip one switch, hold everything else fixed, measure perplexity.**

```
from_scratch: bool    = False    # PHASE SWITCH. False = fine-tune (start here).
                                True = pretrain.
jepa_enabled: bool    = True     # add JEPA loss? False = pure-NTP baseline (the
                                control).
lambda_jepa: float    = 1.0     # weight on JEPA (0.0 behaves like
jepa_enabled=False).
loss_dropout: float   = 0.125   # fraction of JEPA edges randomly skipped (the
                                speed knob).
use_pred_tokens: bool = True     # use [PRED] tokens to read out the prediction
                                vector.
```

Current correct setting for Phase 1: `from_scratch = False`. Do not change it until the decision gate (0.2).

Recommended Phase-1 block in `config.py`:

```
from_scratch = False          # fine-tuning: start from a good model
model_name   = "Qwen/Qwen2.5-0.5B" # open model, no login required
use_lora     = False          # full fine-tuning gives the cleanest
comparison
load_in_4bit = False
```

2.3 How the code flows when you run an experiment

```

experiments.py (you run this)
    for each setting in the study (e.g. each lambda value):
        for each seed:
            → train() (train.py)
                | build_shards() (data_stream.py – runs once, then cached)
                | SciChainJEPA() (model.py – loads PRETRAINED if
from_scratch=False)
                | for each batch:
                    | loss = ntp_loss(full_text) # always on
                    | if jepa_enabled:
                        | for (i,j) in chain_edges:
                            | if random() < loss_dropout: continue # skip some
edges |
                    |
                    | pred = predict_stage(stages 0..i)
                    | tgt = encode(stage j)
                    | loss += lambda_jepa * (1 - cosine(pred, tgt))
                    | backward; clip; optimizer.step(); scheduler.step()
                    | save model + timing stats
            → run_all() (evaluate.py)
                | eval_perplexity() ← THE HEADLINE NUMBER
                | f1_model_retrieval() ; f1_bm25_baseline() ; f2_surprise()
            → results saved to outputs/study_*.json (+ .png plots)

```

Part 3 — Running Experiments

3.1 One-time setup

```
# Put all .py files (and requirements.txt) in one folder, then:
pip install -r requirements.txt
```

Hardware: a single modern GPU (≥ 16 GB) is enough for Phase 1 at the small/P1 tiers. No Hugging Face login is needed (the default model is open).

3.2 The smoke test — always first

```
# ~2 minutes. Runs a tiny version of EVERYTHING to catch errors before you scale
up.
python experiments.py --study all --smoke
```

Rule #1 of this project: *Never launch a long run without passing - - smoke first.* A 2-minute check saves days of wasted GPU time. If smoke fails, copy the **full error message** and bring it to me (or see Part 5).

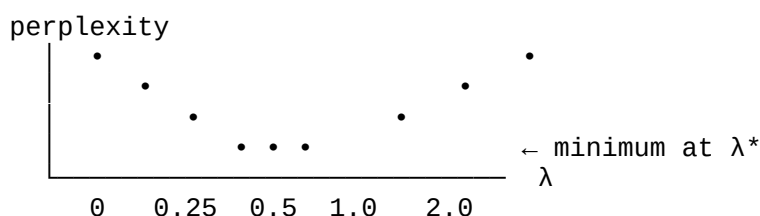
3.3 The three Phase-1 studies

Run them **one at a time** at real scale (each trains many models). Use `--tier P1` for proof-of-concept results, `--tier P2` for larger final runs.

Study 1 — λ -sweep (your first real result; the project's backbone)

```
python experiments.py --study lambda --tier P1 --seeds 3
```

- **Grid:** $\lambda \in \{0, 0.25, 0.5, 1.0, 2.0\}$. $\lambda=0$ is the **baseline** — the sweep contains its own control.
- **Hypothesis:** U-shape — too little JEPA gives no benefit; too much crowds out NTP and hurts. Best at some λ^* .
- **Success criterion:** some $\lambda^*>0$ has perplexity **significantly** below $\lambda=0$ (paired t-test in `stats.py`, Holm-Bonferroni across the grid).



Fill this from `outputs/study_lambda.json`:

λ	Perplexity (mean $\pm$ std)	F1	R@1	F2	p
0.0 (baseline)					
0.25					
0.5					
1.0					
2.0					

Study 2 — Ablation on design choices

```
python experiments.py --study ablation --tier P1 --seeds 3
```

Trains the **full** model, then turns **one design choice off at a time**:

Variant	Removed	Tests
full	nothing	reference point
no_pred_tokens	the [PRED] tokens	do special readout tokens help?
no_long_range_edge	the method $\rightarrow$ result edge	does long-range reasoning matter?
no_trivial_filter	skipping easy edges	does filtering easy edges help?
mse_loss	cosine $\rightarrow$ MSE	which JEPA loss is better?
no_jepa(baseline)	JEPA entirely	the pure-NTP control

The bigger the perplexity jump when a piece is removed, the more it matters.

Study 3 — Faster JEPA via loss dropout

```
python experiments.py --study dropout --tier P1 --seeds 3
```

Sweeps `loss_dropout` $\in \{0, 0.125, 0.25, 0.5, 0.75\}$ and reports **both** perplexity and speed (`steps_per_sec`). The hoped-for story: "*drop X% of edges, train $\sim$ Y% faster, lose almost no quality.*" Plots saved as `study_dropout_*.png`.

3.4 Scaling tiers and cost

Tier	Flag	Scale	Use for
smoke	--smoke	~30 steps, 2k papers	catching errors (~2 min)
P1	--tier P1	proof-of-concept	real Phase-1 results
P2	--tier P2	larger	final Phase-1 / future Phase-2

Cost warning: λ -sweep at P1 with 3 seeds = $5 \times 3 = 15$ **training runs**. First do --seeds 1 to time one run, then scale to --seeds 3. Run studies separately, **not** --study all, at real scale.

3.5 Reading the results

Results land in ./outputs/:

- study_lambda.json, study_ablation.json, study_dropout.json — the numbers.
- study_*.png — the plots.

Use stats.py to test significance between two settings (e.g. λ^* vs. $\lambda=0$) before claiming an improvement.

Part 4 — Modifying the Code

Golden rule: after *any* change, run --smoke again before scaling.

You want to...	Do this
Move Phase 1 → Phase 2	config.py: from_scratch = True. Only after the decision gate (0.2). Also set lr=6e-4, warmup_steps=2000, use_lora=False.
Use a different model	config.py: model_name = "Qwen/Qwen2.5-1.5B" (etc.). Open models need no login.
Train on more data	config.py: raise max_train_examples, max_train_steps; or use --tier P1/P2.
Change the reasoning chain	config.py: edit stages and stage_columns (names must match dataset columns).
Add / remove a JEPA edge	config.py: edit chain_edges (e.g. add (1, 3)).
Try a different λ grid	experiments.py: edit lambdas = [...] in study_lambda.
Add an ablation variant	experiments.py: add an entry to the variants dict in study_ablation.
Change the JEPA loss	config.py: jepa_loss_type = "mse" (or "cosine"); logic in train.py: jepa_loss.
Save GPU memory (fine-tuning)	config.py: use_lora = True (cleaner comparison without it — ask me first).
Run on multiple GPUs	accelerate config once, then accelerate launch experiments.py

You want to...	Do this
Resume after a crash	<code>config.py: resume_from = "outputs/step5000".</code>
Change the train/test split	<code>config.py: test_year, train_cutoff_year, random_split_test_frac.</code>
Example — adding a new chain edge:	

```
# config.py
chain_edges = [(0,1),(1,2),(2,3),(3,4),(4,5),(2,4),
               (1,4)] # NEW: hypothesis → result (very long-range)
```

Run `--smoke`, confirm it runs, then study its effect via the ablation framework.

Part 5 — Troubleshooting

Symptom	Likely cause	Fix
CUDA out of memory	batch too big / model too big	lower <code>batch_size</code> , raise <code>grad_accum_steps</code> , enable <code>gradient_checkpointing</code> , or set <code>use_lora=True</code> .
Smoke test errors immediately	install or dataset issue	re-run <code>pip install -r requirements.txt</code> ; check internet for dataset download; paste full traceback to me.
<code>KeyError</code> on a stage column	dataset column names changed	fix <code>stage_columns</code> in <code>config.py</code> to match the dataset.
Perplexity is <code>nan</code> / <code>inf</code>	LR too high / instability	lower <code>lr</code> ; ensure <code>warmup_steps > 0</code> ; check <code>clip_grad_norm</code> .
Training extremely slow	too many JEPA edges per step	raise <code>loss_dropout</code> , lower <code>num_pred_tokens</code> , or reduce <code>chain_edges</code> .
<code>bitsandbytes</code> import error	4-bit enabled without the lib	<code>pip install bitsandbytes</code> , or set <code>load_in_4bit=False</code> .
Results look random / no effect	seeds too few / scale too small	run more <code>--seeds</code> ; move from <code>--smoke</code> to <code>--tier P1</code> .

When in doubt: **run `--smoke`, read the full traceback, and bring the exact command + error to me.** Don't debug a 12-hour run; reproduce it in the 2-minute smoke run first.

Part 6 — Reference

6.1 Honest caveats (please read before final runs)

1. **Phase 1 (fine-tuning) is a complete, publishable result.** You are not obligated to reach Phase 2.
2. **Phase 2 (from scratch) needs far more data than we have.** Our corpus is short structured summaries; a from-scratch model on it will be a weak generator with high *absolute* perplexity. Only the *relative* comparison is meaningful, and even

that needs more data to be credible. We'd likely need the full-text column or external corpora first.

3. **The streaming shuffle is approximate** (buffer-based, not global). Fine for training; just know it.
4. **The temporal split may fall back to a hash split** if 2025 data is sparse. Decide and *state* your split before final runs.
5. **Per-edge forward passes are not fused** (~6× JEPA compute). Loss dropout mitigates it; true fusion is a future engineering task.
6. **The title must match what we ran.** If we only do Phase 1, the paper says "fine-tuning," not "pretraining." We choose the title *after* seeing results.

6.2 Key config fields

Field	Meaning	Typical
from_scratch	Phase switch (fine-tune vs. pretrain)	False (Phase 1)
jepa_enabled	add JEPA loss	True
lambda_jepa	weight on JEPA	swept
loss_dropout	fraction of edges skipped	swept
use_pred_tokens	use [PRED] readout tokens	True
model_name	base model	Qwen/Qwen2.5-0.5B
chain_edges	which stage predicts which	see 1.3
batch_size / grad_accum_steps	effective batch = product	tier-dependent
lr / warmup_steps	learning rate / warmup	6e-4 / 2000 (scratch)
max_train_steps / max_train_examples	training length / data	tier-dependent
seed	RNG seed	swept via --seeds

6.3 Glossary

Term	Meaning
Phase 1 / Phase 2	Our plan: fine-tuning first; pretraining-from-scratch only if earned.
from_scratch	False = start from pretrained weights (fine-tune); True = random init (pretrain).
NTP	Next-token prediction — the standard "predict the next word" objective.
JEPA	Predict the <i>embedding</i> of what comes next, not the exact tokens.
Perplexity	exp(avg next-token loss). Lower = better generator. The headline metric.
Embedding	A vector capturing the meaning of a piece of text.
Representation collapse	Failure mode where all embeddings become identical (useless). NTP prevents it.
Loss dropout	Randomly skipping some JEPA edges per step to save compute.
Ablation	Removing one component to measure its contribution.
Edge	A (from_stage, to_stage) pair the JEPA objective predicts across.
Seed	RNG starting point; multiple seeds ensure results aren't luck.
λ (lambda_jepa)	Weight on the JEPA loss in the combined objective.

Term	Meaning
F1 / F2	Diagnostic tasks: forward retrieval / surprise calibration.

6.4 Pre-flight checklist (before any real run)

- `from_scratch = False` (Phase 1) confirmed in `config.py`.
- `python data_stream.py` prints a sensible example chain.
- `python experiments.py --study all --smoke` passes with no error.
- One `--seeds 1` run timed, so you know the cost of `--seeds 3`.
- Disk space for `outputs/` and `data_shards/`.
- Running **one study at a time**, not `--study all`, at real scale.

6.5 Suggested first two weeks

1. **Day 1** — Read Parts 0–1. Run `python data_stream.py`; inspect the example chain.
2. **Day 2** — Confirm `from_scratch=False`. Pass `--smoke`. Read an `outputs/study_*.json`.
3. **Day 3** — Read `train.py` + `model.py` alongside 2.3. Trace one batch by hand.
4. **Day 4–5** — Launch the λ -sweep at `--tier P1` (start `--seeds 1`, then 3). Prepare the results table.
5. **Week 2** — Run the **ablation** and **dropout** studies. Use `stats.py` for significance.
6. **End of Week 2** — Bring me the λ plot + perplexity table. **We make the Phase 1 → Phase 2 go/no-go decision together.**